A Two-Stage Building Damage Assessment Pipeline for Post-Earthquake Imagery: Accuracy-Latency Tradeoffs and Cross-Domain Evaluation Research

Mustafa Eren Soyhan
Student ID: 101045056
Electrical and Computer Engineering, Ontario Tech University
mustafa.soyhan@ontariotechu.net

***Abstract*—This project develops and evaluates a two-stage building damage assessment pipeline for post-earthquake imagery. The first stage uses a YOLO-based detector for building localization, while the second stage employs a lightweight classifier for damage severity prediction. Using the earthquake subset of the xBD dataset for training and evaluation, the project systematically benchmarks accuracy-latency tradeoffs across model sizes, input resolutions, and inference precision. Additionally, the pipeline is evaluated on the UAVs-TEBDE dataset to assess cross-domain robustness when transferring from satellite imagery to UAV imagery. The expected contribution is a reproducible evaluation framework that jointly reports detection quality, classification accuracy, runtime metrics, and cross-domain degradation patterns.**

***Index Terms*—earthquake damage assessment, building detection, image classification, YOLO, latency benchmarking, cross-domain evaluation**

I. Introduction

Post-earthquake damage assessment is time-critical for emergency response and resource allocation. While deep learning models achieve strong performance on benchmark datasets, practical deployments must also satisfy runtime constraints and maintain robustness under varying imaging conditions. These factors create a gap between research results and deployment-ready systems.

Building damage assessment from imagery involves two distinct sub-tasks: (1) localizing buildings within an image, and (2) classifying the damage severity of each detected building. Many existing approaches address these jointly through semantic segmentation [1], but this coupling makes it difficult to independently evaluate and optimize each component. Furthermore, models trained on satellite imagery may not transfer well to UAV imagery due to differences in viewpoint, resolution, and scene composition [5].

This project addresses these challenges through a modular two-stage pipeline that separates building localization from damage classification. This design enables independent benchmarking of each stage, facilitates component-level optimization, and provides clearer insight into where performance bottlenecks occur. The project evaluates accuracy-latency tradeoffs across practical configurations and quantifies cross-domain degradation when transferring from benchmark satellite imagery to UAV imagery collected after real earthquake events.

II. Problem Definition and Scope

Building damage assessment pipelines are typically evaluated on benchmark accuracy alone, while practical deployment requires predictable latency and robustness to domain shift. When models trained on satellite imagery are applied to UAV imagery, differences in perspective and image characteristics can substantially degrade performance. This gap between benchmark evaluation and real-world deployment motivates the need for a more comprehensive assessment framework.

This project investigates three research questions: first, what accuracy-latency tradeoffs exist across model size and input resolution configurations for building localization and damage classification; second, how inference precision (FP32 vs. FP16) affects accuracy and runtime for each pipeline stage; and third, how much performance degrades when applying satellite-trained models to UAV imagery, and what failure patterns emerge during this domain transfer.

The project uses the earthquake subset of xBD [1] for training and primary evaluation, focusing on earthquake-specific damage patterns rather than all disaster types. Cross-domain evaluation uses the publicly available UAVs-TEBDE dataset [2], which provides UAV imagery from the 2023 Türkiye earthquakes. The implementation uses PyTorch [4] with the Ultralytics YOLO framework [3] for detection and standard classifier architectures for damage classification.

III. Methodology

The methodology follows a systematic approach where each pipeline component is independently implemented, trained, and evaluated before integration. This section describes the two-stage architecture, dataset preparation, experimental configurations, and evaluation protocols.

A. Two-Stage Pipeline Architecture

The pipeline separates building damage assessment into two sequential stages, as illustrated in Figure 1.

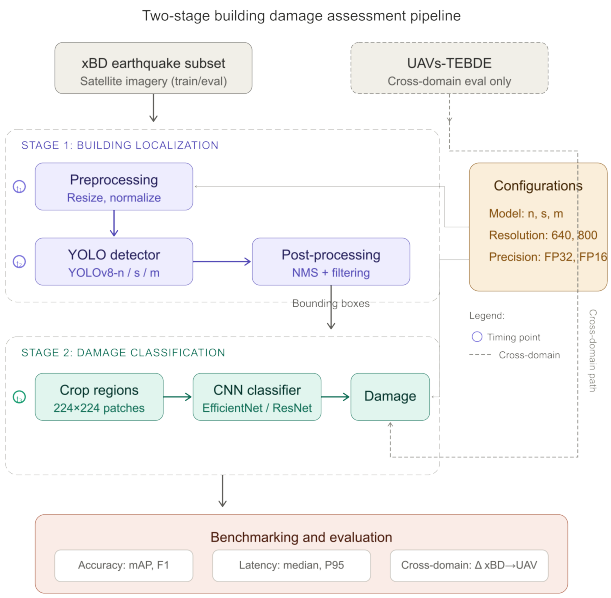

Fig. 1. Two-stage damage assessment pipeline. Stage 1 localizes buildings using YOLO-based detection. Stage 2 classifies damage severity for each detected region using a lightweight CNN classifier. Timing points (t_1, t_2, t_3) indicate where latency is measured. The dashed path shows cross-domain evaluation flow from UAVs-TEBDE.

The first stage performs building localization using a YOLO-based object detector that identifies building locations in post-disaster imagery, outputting bounding boxes with confidence scores. This stage treats building detection as a single-class problem, deferring damage assessment to the second stage. The detector is trained on building polygons from xBD converted to bounding box format.

The second stage handles damage classification, where a CNN-based classifier predicts damage severity for each detected building region. The classifier receives cropped and resized image patches corresponding to detected buildings and outputs a damage level prediction. This modular design allows the classifier to be trained and evaluated independently of the detector, following established multi-stage approaches for disaster damage assessment [6].

During inference, Stage 1 processes the full image to produce building detections. Each detection above a confidence threshold is cropped from the original high-resolution image, resized to a fixed input size, and passed to the Stage 2 classifier. The final output combines localization (bounding box coordinates) with classification (damage level) for each building. Algorithm 1 formalizes the inference procedure with timing instrumentation.

B. Dataset Preparation

The xBD dataset contains pre- and post-disaster satellite imagery across 19 disaster events with building polygon annotations and four-level damage labels (no-damage, minor, major, destroyed) [1]. This project uses only the earthquake-related events to maintain domain consistency

Algorithm 1 Two-Stage Inference with Timing

Require: Image I, detector D, classifier C, thresholds τ_{conf}, τ_{nms}
Ensure: Detections with damage labels, timing metrics

$t_0 \leftarrow \textsc{Time}()$
$I' \leftarrow \textsc{Preprocess}(I)$
$t_1 \leftarrow \textsc{Time}()$
$boxes \leftarrow D(I')$ ▷ Stage 1: Detection
$boxes \leftarrow \text{NMS}(boxes, \tau_{conf}, \tau_{nms})$
$t_2 \leftarrow \textsc{Time}()$
$results \leftarrow \emptyset$
for each $box \in boxes$ **do**
  $patch \leftarrow \textsc{Crop}(I, box)$
  $damage \leftarrow C(patch)$ ▷ Stage 2: Classify
  $results \leftarrow results \cup \{(box, damage)\}$
end for
$t_3 \leftarrow \textsc{Time}()$
return $results$, $(t_1 - t_0, t_2 - t_1, t_3 - t_2)$

and reduce training scope. Building polygons are converted to axis-aligned bounding boxes for detector training, while classifier training uses building regions cropped from post-disaster images using polygon bounds and labeled with the corresponding damage level.

The UAVs-TEBDE dataset provides UAV imagery from the 2023 Türkiye earthquakes with three damage categories: Collapsed, Damaged, and Intact [2]. This dataset serves exclusively for cross-domain evaluation, with no training performed on UAVs-TEBDE. To enable comparison across datasets, the xBD four-class labels are mapped to three classes: xBD *no-damage* maps to *Intact*, xBD *minor-damage* and *major-damage* merge into *Damaged*, and xBD *destroyed* maps to *Collapsed*. This harmonization is applied during evaluation only and does not affect training on the original four-class xBD labels.

C. Experimental Configurations

The experiment systematically varies configuration parameters to characterize accuracy-latency tradeoffs, as summarized in Table I. Three YOLO model sizes (nano, small, medium) represent different capacity-speed tradeoffs, while input resolutions of 640 and 800 pixels balance detail capture against computational cost. For classification, two lightweight architectures are evaluated: EfficientNet-B0 for its efficient mobile-oriented design and ResNet-34 as an established baseline, both using 224×224 input patches. All configurations are evaluated in both FP32 (standard) and FP16 (mixed precision) modes to assess the accuracy-speed tradeoff of reduced precision inference.

D. Training Protocol

Training uses consistent hyperparameters across configurations to ensure fair comparison, with the goal of evaluating pipeline tradeoffs rather than maximizing individual

TABLE I
EXPERIMENTAL CONFIGURATION SPACE

Component	Parameter	Values
Detector	Model size	YOLOv8-n, -s, -m
	Resolution	640, 800
Classifier	Architecture	EfficientNet-B0, ResNet-34
	Input size	224
Both	Precision	FP32, FP16

benchmark scores. YOLO models are trained on the xBD earthquake subset with standard augmentation including mosaic, mixup, and geometric transforms, using a fixed epoch count with early stopping based on validation mAP. Batch size is set based on available GPU memory for each model size.

Classifiers are initialized with ImageNet-pretrained weights and fine-tuned on cropped building patches from xBD. Training uses cross-entropy loss with class weighting to address damage-level imbalance, and a fixed learning rate schedule with warm-up and cosine decay is applied across all configurations. All training runs log random seeds, dataset splits, and hyperparameters, with model checkpoints saved at best validation performance for subsequent evaluation.

E. Experimental Setup

All experiments are implemented in Python using PyTorch 2.x with CUDA 12.x, with detection models from Ultralytics YOLOv8 and classification using EfficientNet-B0 [7] and ResNet-34 via torchvision. The hardware platform consists of an NVIDIA RTX 4080 Laptop GPU (12GB VRAM), 32GB system RAM, and an Intel Core i9-13900HX processor. All code and trained weights will be released for reproducibility.

F. Evaluation Protocols

Detector performance is measured using mean Average Precision (mAP@0.5 and mAP@0.5:0.95), while classifier performance is measured using accuracy, macro-averaged F1-score, and per-class precision/recall. For the integrated pipeline, we report the combined metric of correctly localized and correctly classified buildings.

To ensure fair timing comparisons, the evaluation protocol includes a warm-up phase of 10 inference passes before timing to initialize GPU caches, repeated evaluation where each image is processed multiple times with median latency reported, stage-level timing with separate measurements for preprocessing, detection, and classification, and percentile reporting including both median and 95th percentile (P95) latency to capture tail behavior. Throughput is reported as images per second for the full pipeline, and hardware and software versions are documented for reproducibility.

Results are visualized as tradeoff plots where each configuration is a point with accuracy on one axis and latency on the other. The analysis identifies Pareto-optimal configurations—those not dominated by any other configuration in both metrics—and highlights practical operating points for different latency budgets.

For cross-domain evaluation, the trained classifier is applied directly to UAVs-TEBDE without fine-tuning to measure transfer performance. Since UAVs-TEBDE provides pre-cropped building images, only the Stage 2 classifier is evaluated for cross-domain transfer. The evaluation reports classification accuracy using the label mapping, performance delta between xBD validation and UAVs-TEBDE, and confusion matrix analysis to identify systematic failure patterns. This evaluation is explicitly exploratory; we document the magnitude and nature of domain shift rather than expecting strong transfer performance.

G. End-to-End Example

To illustrate the complete workflow with estimated timings, consider processing a single 1024×1024 pixel post-earthquake satellite image from the xBD test set. In Stage 1, the image is resized to 640×640, normalized, and passed through YOLOv8-s. The detector outputs 12 candidate boxes; after NMS with $\tau_{conf} = 0.25$ and $\tau_{nms} = 0.5$, 8 building detections remain, estimated at approximately 18ms based on typical YOLOv8-s throughput. In Stage 2, each of the 8 detected regions is cropped from the original image, resized to 224×224, and classified by EfficientNet-B0, yielding predictions of 2 no-damage, 3 minor, 2 major, and 1 destroyed building, estimated at approximately 24ms total. The complete pipeline produces 8 buildings with bounding boxes and damage labels in an estimated 42ms (23.8 fps). Repeating this process across the test set produces dataset-level metrics and latency statistics that populate the configuration comparison.

IV. EXPECTED CONTRIBUTIONS

This project delivers three contributions. First, a modular two-stage pipeline for building damage assessment that separates localization and classification, enabling independent optimization and clearer performance analysis. The implementation includes timing instrumentation and configuration management for reproducible experiments.

Second, an accuracy-latency tradeoff characterization showing empirical results for how model size, input resolution, and inference precision jointly affect detection quality, classification accuracy, and runtime. The analysis identifies Pareto-optimal configurations suitable for different deployment constraints.

Third, a cross-domain transfer analysis documenting performance degradation when applying satellite-trained classifiers to UAV imagery, including quantified accuracy drops and categorized failure patterns. This analysis provides realistic expectations for domain transfer in earthquake damage assessment.

These contributions emphasize practical system behavior and reproducible evaluation over maximizing single-dataset benchmarks.

References

[1] R. Gupta, B. Goodman, N. Patel, R. Hosfelt, S. Sajeev, E. Heim, J. Doshi, K. Lucas, H. Choset, and M. Gaston, "Creating xBD: A dataset for assessing building damage from satellite imagery," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019, pp. 10–17.

[2] M. N. Mowla and D. Asadi, "UAVs-based Turkey Earthquake Building Damage Estimation Dataset (UAVs-TEBDE)," Mendeley Data, V1, 2025. [Online]. Available: https://doi.org/10.17632/5m349hfvkb.1

[3] Ultralytics, "Ultralytics YOLO," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[4] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] S. Tilon, F. Nex, N. Kerle, and G. Vosselman, "Post-disaster building damage detection from earth observation imagery using unsupervised and transferable anomaly detecting generative adversarial networks," *Remote Sens.*, vol. 12, no. 24, p. 4193, 2020.

[6] E. Weber and H. Káné, "Building disaster damage assessment in satellite imagery with multi-temporal fusion," in *Proc. IEEE/CVF Int. Conf. Computer Vision Workshops (ICCVW)*, 2020, pp. 4571–4580.

[7] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019, pp. 6105–6114.